

Scalable Updatable Secure Storage from Linear-Size Homomorphic Vector Commitments

Long Chen Shuangyu He

Institute of Software, Chinese Academy of Sciences, Beijing 100190

chenlong@iscas.ac.cn, heshuangyu@iscas.ac.cn

Abstract

Updatable secure storage (USS) enables a client to outsource a dynamic encrypted database, maintain a single short local stub, and detect a malicious server that serves stale files—even after current secret keys leak. To allow server-side key rotation without downloading and re-encrypting the data, USS relies on updatable encryption (UE). However, existing USS constructions remain theoretical and inefficient in practice. The only published construction (Chen–Guo–Li–Tang, ASIACRYPT 2023) relies on a Catalano–Fiore homomorphic vector commitment (HVC) with an $O(n^2)$ common reference string (tens of megabytes at $n = 2^{10}$ and prohibitive at $n = 2^{14}$), while leaving the data-encryption mechanism (DEM) unmeasured for large files.

This paper makes USS practical by resolving both bottlenecks. First, we present KHVC, a KZG-based HVC featuring an $O(n)$ SRS, tight q -SDH position binding, and statistical position hiding. At $n = 1024$, parameter generation time drops from 70.2 s to 0.38 s, and operational parameters decrease from 50.5 MB to 96.3 KB. Second, since CGLT23 allows any IND-CPA UE as the DEM (as ciphertext integrity is guaranteed via the stub), we instantiate a random-oracle Ring-LWR key-homomorphic PRF sized for $t \leq 127$ correct updates, caching public a_b NTTs to optimize update performance. FileUp openings use a one-hot closed form to eliminate dense FK20 overhead. We implement our scheme and the 2023 baseline in Rust (BLS12-381) and evaluate them on files up to 1 GB. On an Apple M5 Pro, at 1 MB the DEM achieves 32.6 MB/s with a $2\times$ ciphertext expansion and a 4096-byte token.

Keywords: vector commitment, KZG, updatable encryption, updatable secure storage, key rotation.

1 Introduction

Cloud storage services are increasingly widely used for outsourcing massive dynamic databases. To meet standard security policies (e.g., PCI DSS and NIST guidelines) requiring periodic cryptographic key rotation, *Updatable Encryption* (UE) [4, 5, 7, 6] has emerged as a primitive of choice. UE allows a storage server to re-encrypt ciphertexts to a fresh epoch using a light update token generated by the client, without learning the underlying plaintext or requiring the client to download and re-upload the entire dataset.

However, standard UE alone is insufficient for securing dynamic database environments. A malicious or compromised server, even when performing key rotations correctly, can still mount a state-rollback attack by returning stale ciphertexts from earlier epochs that successfully decrypt under the current key. To enforce version integrity and detect such rollback behavior, Chen, Guo, Li, and Tang [12] (*Efficient Secure Storage with Version Control and Key Rotation*, ASIACRYPT 2023) formalize *Updatable Secure Storage* (USS). In a USS scheme, the client maintains only a single short, constant-size local stub, while the update communication for file and key updates remains independent of the total file count n . Most importantly, USS guarantees authenticity and freshness detection even if current secret keys have been leaked.

To achieve this, CGLT23 proposed a modular KEM+DEM blueprint: a Homomorphic Vector Commitment (HVC) commits to file hashes to form the stub, an updatable KEM (e.g., RISE) protects evaluation openings, and a ciphertext-independent UE acts as the Data Encryption Mechanism (DEM) for payload files.

Motivation & Limitations of Prior Art. Despite its strong theoretical security, existing USS constructions remain far from practical deployment for real-world, large-scale file systems. Specifically, prior work suffers from two major computational and storage bottlenecks:

1. **Explosive SRS/CRS Scale in HVC:** The baseline USS construction [12] relies on the Catalano–Fiore (CF) HVC [2]. Because CF-HVC requires precomputing every pairwise group product $g^{z_i z_j}$, its Common Reference String (CRS) grows quadratically as $O(n^2)$. In practice, for a database of $n = 2^{10}$ files, the parameters exceed 50 MB, and setup time requires over a minute; at $n = 2^{14}$, the parameters quickly balloon to several gigabytes, rendering setup and operation prohibitively expensive.
2. **Unrealized DEM Efficiency for Large Payloads:** While the CGLT23 theoretical framework allows any IND-CPA UE as the DEM (since file integrity is verified via the local stub rather than the DEM itself), prior literature leaves the actual performance of payload encryption unmeasured and unoptimized for large files. Wrapping gigabyte payloads with traditional group-based or curve-based key-homomorphic PRFs introduces unbearable performance degradation and massive ciphertext expansion, while standard symmetric modes like AES-GCM lack native update mechanisms.

1.1 Our Contributions.

In this paper, we bridge the gap between theoretical USS definitions and real-world performance by designing, instantiating, and measuring a fully practical, high-throughput USS system for large-file databases. Our main contributions are summarized as follows:

- **Linear-Size HVC (KHVC):** We introduce KHVC, a succinct KZG-based homomorphic vector commitment with an $O(n)$ Structured Reference String (SRS). KHVC achieves tight q -SDH position binding and statistical position hiding with a succinct auxiliary string $\text{aux} = \rho \in \mathbb{F}_p$. By deriving a closed-form formula for single-slot updates, we eliminate dense FK20 evaluation overhead during FileUp operations. Compared to CF-HVC at $n = 1024$, KHVC reduces parameter size from 50.5 MB to 96.3 KB (over $500\times$ reduction) and setup time from 70.2 s to 0.38 s.
- **High-Throughput Ring-LWR DEM:** Exploiting the insight that USS requires only IND-CPA security for the DEM, we instantiate an efficient lattice-based DEM using a random-oracle Ring-LWR key-homomorphic PRF sized for up to 127 consecutive key updates (sufficient for decades of periodic rotation). By pre-caching public NTT representations and removing redundant header embeddings from the body ciphertexts, we achieve near-native payload update speeds with moderate expansion.
- **Implementation & End-to-End Evaluation:** We implement the complete USS architecture alongside the 2023 baseline in Rust using BLS12-381. On an Apple M5 Pro workstation, our Ring-LWR DEM achieves a re-encryption throughput of 32.6 MB/s on 1 MB payloads with a $2\times$ ciphertext expansion, demonstrating for the first time that USS can scale efficiently to gigabyte-sized cloud databases.

1.2 Related Work

Vector and Polynomial Commitments. Vector Commitments (VCs) [2, 14, 13] and Polynomial Commitments (PCS) [3] form foundational building blocks in modern cryptography. While recent works adapt KZG commitments as vector commitments—for instance, Fleischhacker et al. [16] employ hidden KZG for extractable witness encryption, and Lasso [17] utilizes PCS within proof systems—these constructions do not meet the structural requirements of Updatable Secure Storage (USS). Specifically, they omit additively homomorphic operations (**ComHom** and **OpenHom**) and fail to maintain an $O(1)$ auxiliary string **aux** independent of the vector dimension n . Furthermore, they do not guarantee position hiding when slot openings are exposed as group elements, which is essential for client-side privacy in key updates.

Transparent and Post-Quantum Alternatives. Transparent polynomial commitments based on FRI [18] eliminate the need for a trusted setup and pairing-friendly curves. However, FRI openings consist of Merkle authentication paths rather than algebraic group elements in $\mathbb{G}_1$. Consequently, FRI-based commitments cannot directly interface with algebraic homomorphic encryptions (such as RISE [4]) to support secure update token evaluations, necessitating a pairing-based KZG approach for our HVC instantiation.

Updatable Encryption and Data Encapsulation. Updatable Encryption (UE) schemes allow untrusted servers to rotate ciphertexts across key epochs using re-encryption tokens [5, 4, 6]. To scale UE to bulk payloads, Boneh et al. [7] explored Key-Homomorphic PRFs (KH-PRFs), moving from Ring-LWR to RLWE to expand the noise budget for unbounded updates. In contrast, we observe that under a practical ≤ 127 -update lifecycle (consistent with NIST key rotation guidelines [11]), a Ring-LWR DEM provides sufficient noise capacity without incurring RLWE’s performance overhead.

Updatable Secure Storage. The USS framework introduced by CGLT23 [12] provides end-to-end data privacy and integrity against malicious servers. However, their concrete instantiation relies on Catalano–Fiore HVC, which suffers from a quadratic CRS size ($O(n^2)$) and renders large-scale deployments intractable. This work delivers the first linear-SRS ($O(n)$) HVC instantiation for USS, paired with an empirically measured large-file DEM for efficient bulk data rotation.

2 Preliminaries

2.1 Notation and Terminology

Let $\mathbb{F}_p$ denote the scalar field of the BLS12-381 elliptic curve (the group order). For an element $a \in \mathbb{F}_p$, we write $[a]_1 = ag_1 \in \mathbb{G}_1$ and $[a]_2 = ag_2 \in \mathbb{G}_2$ for fixed generators g_1, g_2 . USS file slots are indexed $i \in \{1, \dots, n\}$; when interacting with KHVC (whose native indices run over $\{0, \dots, n-1\}$) we identify slot i with root ω^{i-1} . UE epochs in Section 4 are numbered from 0 as in [4]; USS epochs in Section 5 start at 1 as in [12]. Throughout we distinguish:

- $\rho \in \mathbb{F}_p$: the blinding scalar of KHVC.
- $r \in \mathbb{F}_p$: ephemeral RISE encryption randomness.
- $\nu_i \in \{0, 1\}^{256}$: the 256-bit nonce bound into the file commitment at slot i .
- The integer q in the q -SDH assumption equals the polynomial degree bound n ; it is *unrelated* to the Ring-LWR modulus $q = 998244353$ of Section 4.
- The BLS scalar field $\mathbb{F}_p$ is *unrelated* to the Ring-LWR rounding modulus $p = 2^{16}$ of Section 4; the overload is confined to that section, where no pairing field appears.

2.2 Pairings and Hardness Assumptions

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be an asymmetric Type-3 bilinear pairing on BLS12-381. The security of our constructions relies on the following standard assumptions.

Assumption 1 (Symmetric External Diffie-Hellman, SXDH). *The Decisional Diffie-Hellman (DDH) problem is computationally hard independently in both $\mathbb{G}_1$ and $\mathbb{G}_2$.*

Assumption 2 (q -Strong Diffie-Hellman, q -SDH [1, 3]). *Given a $(q+3)$ -tuple $([1]_1, [x]_1, \dots, [x^q]_1, [1]_2, [x]_2)$ for a uniformly sampled secret $x \leftarrow \mathbb{F}_p^*$, no probabilistic polynomial-time (PPT) adversary can output a tuple $(c, [1/(x+c)]_1)$ with $c \neq -x$ except with negligible probability. In our reduction, this tuple instantiates the KHVC Structured Reference String (SRS) of degree $q = n$.*

2.3 Underlying Cryptographic Primitives

KZG Polynomial Commitments [3]. The $\text{KZG.Setup}(1^\lambda, d)$ algorithm samples $\tau \leftarrow \mathbb{F}_p^*$ and publishes $([1]_1, \dots, [\tau^d]_1, [1]_2, [\tau]_2)$. A polynomial $f(X) \in \mathbb{F}_p[X]$ is committed as $C = [f(\tau)]_1$. An evaluation proof at point z is given by $\Lambda = [q(\tau)]_1$, where $q(X) = (f(X) - f(z))/(X - z)$. Evaluation binding reduces directly to q -SDH. Commitments and evaluation proofs inherit additive homomorphism from the exponentiation operation. Notice that verification requires only $[1]_2$ and $[\tau]_2$, eliminating the need for $[\tau^d]_2$.

RISE Updatable Encryption [4]. RISE is an ElGamal-style Updatable Encryption (UE) scheme over a prime-order group:

- $\text{Enc}_k(M) = (rY, rG + M)$ under public key $Y = kG$ and ephemeral scalar $r \leftarrow \mathbb{F}_p$.
- Given a transition token $(\Delta = k'/k, Y' = k'G)$ and fresh $r' \leftarrow \mathbb{F}_p$, the server updates $C = (C_1, C_2)$ to

$$C' = (\Delta \cdot C_1 + r'Y', C_2 + r'G).$$

Decryption under k' recovers M because $\Delta \cdot C_1 = rY'$ and thus $C'_2 - (k')^{-1}C'_1 = M$.

Lehmann and Tackmann proved that RISE achieves IND-ENC and rand-IND-UPD security under DDH. We instantiate RISE in $\mathbb{G}_1$ under SXDH. Because RISE is additively homomorphic over $\mathbb{G}_1$, it encapsulates HVC openings; it is not intended for large payloads.

Updatable Secure Storage (USS) Framework [12]. An Updatable Secure Storage (USS) system manages a database of n outsourced files across client key update epochs. USS.Store generates a ciphertext repository and a small local client stub C ; USS.Rev retrieves and verifies file i ; USS.FileUp updates single file contents; and USS.KeyUp performs server-side key rotations. We adopt the standard security definitions from CGLT23 [12]: IND-DD-UP (Data Privacy under Direct Updates), IND-FileUp-CPA (Privacy under File Updates), IND-REENC-CPA (Privacy under Key Rotation), and OF-PTXT (Out-of-Date Plaintext Authenticity against a malicious server possessing leaked secret keys).

2.4 System Instantiation Hypotheses

To ensure complete self-containment, the full protocol instantiation and formal proofs in subsequent sections rest upon the following hypotheses:

1. **Pairing Environment:** Asymmetric Type-3 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ over BLS12-381, assuming SXDH (Assumption 1).
2. **Vector Commitment SRS:** A trusted KZG SRS generated under Assumption 2 with $q = n$, where knowledge of toxic waste τ is destroyed immediately after setup.

3. **Random Oracles:** SHA-256 modeled as a Random Oracle, both for commitment digest generation $\text{COM}(m_i; \nu_i)$ and public ring element expansion $a_b = H(\text{tweak} \parallel b)$.
4. **Entropy and UE:** Nonces $\nu_i \in \{0, 1\}^{256}$ with full 256-bit entropy; RISE instantiated in $\mathbb{G}_1$ under SXDH [4].
5. **Ring-LWR Parameters:** Decisional R-LWR over polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ with dimension $N = 1024$, modulus $q = 998244353$, rounding modulus $p = 2^{16}$, and a uniform secret $s \leftarrow R_q$.
6. **Noise Bound for DEM/UE:** The Ring-LWR DEM supports up to $t \leq 127$ consecutive key rotations. Each key update introduces a rounding noise bounded by $\|e\|_\infty \leq 1$, and 8 reserved noise bits per coefficient absorb a maximum cumulative absolute error of 127.

3 KHVC: Linear-Size Homomorphic Vector Commitment

A Homomorphic Vector Commitment (HVC) [12] allows a client to commit to a vector of messages $\vec{m} = (m_0, \dots, m_{n-1}) \in \mathbb{F}_p^n$ via a succinct string C , generate openings Λ_i proving that m_i is the i -th entry, and dynamically update C and Λ_i as vector elements change. In the context of USS, HVC forms the core of the client’s local stub to enforce version freshness.

However, existing HVC instantiations impose severe system overhead. The standard construction proposed by Catalano and Fiore (CF-HVC) [2] relies on precomputing all pairwise group products $g^{z_i z_j}$, resulting in a Common Reference String (CRS) of size $O(n^2)$. As n scales to tens of thousands of files, this quadratic growth causes parameter generation times to explode into minutes and parameter sizes into gigabytes, creating a critical bottleneck for system startup and update operations.

Design Strategy & Technical Intuition. To overcome the quadratic parameter explosion while retaining strong position hiding and binding, we design KHVC, a succinct $O(n)$ -size HVC built upon the Kate–Zaverucha–Goldberg (KZG) polynomial commitment framework [3]. Our optimization is driven by three key insights:

1. **Linear-Size SRS via Roots of Unity:** We evaluate polynomials over the domain $\Omega = \{\omega^0, \dots, \omega^{n-1}\}$, defined by the n -th roots of unity, with the vanishes-on- Ω polynomial $Z(X) = X^n - 1$. Under this setting, standard KZG opening proofs need only a degree- n Structured Reference String (SRS) consisting of $n + 1$ elements in $\mathbb{G}_1$ and 2 elements in $\mathbb{G}_2$, cutting SRS complexity from $O(n^2)$ down to $O(n)$.
2. **Zero-Overhead Statistical Position Hiding:** To satisfy the privacy requirement that an opening Λ_i reveals no information about other vector slots $m_{j \neq i}$, we inject a random blinding scalar $\rho \leftarrow \mathbb{F}_p$ by defining $F(X) = f(X) + \rho Z(X)$, where $f(\omega^i) = m_i$. Since $Z(\omega^i) = 0$ for all $\omega^i \in \Omega$, adding $\rho Z(X)$ leaves evaluation results $F(\omega^i) = m_i$ completely intact. Consequently, position hiding is achieved statistically via a single scalar ρ , eliminating the need for additional $\mathbb{G}_2$ elements in the SRS.
3. **One-Hot Closed Form for Fast Updates:** Instead of performing dense $O(n \log n)$ FK20 proofs [15] over all n slots during single-slot file updates (FileUp), we precompute a key update table $W_j = [Z(\tau)/(\tau - \omega^j)]_1$ via a single $\mathbb{G}_1$ FFT during setup. We derive a closed-form formula for one-hot vector deltas, reducing off-slot opening updates to simple $O(1)$ scalar multiplications.

3.1 HVC Definitions

Syntax. A Homomorphic Vector Commitment (HVC) scheme for vectors of dimension n consists of six algorithms:

- **HVC.Setup**($1^\lambda, n$) $\rightarrow$ **pp**: Takes as input the security parameter 1^λ and vector size n , and outputs public parameters **pp**.
- **HVC.Com**(**pp**, $\vec{m}$) $\rightarrow$ (C , **aux**): Takes as input **pp** and a message vector $\vec{m} = (m_0, \dots, m_{n-1}) \in \mathcal{M}^n$, and outputs a succinct commitment C along with auxiliary information **aux** (where $|\mathbf{aux}|$ is independent of n).
- **HVC.Open**(**pp**, $\vec{m}$, **aux**, i) $\rightarrow \Lambda_i$: Takes as input **pp**, vector $\vec{m}$, auxiliary string **aux**, and index $i \in \{0, \dots, n-1\}$, and outputs an opening proof Λ_i proving that the i -th entry is m_i .
- **HVC.Ver**(**pp**, C , i , m_i , Λ_i) $\rightarrow 1/0$: Takes as input **pp**, commitment C , index i , message m_i , and proof Λ_i , and outputs 1 if the proof is valid, or 0 otherwise.
- **HVC.ComHom**(**pp**, C , C') $\rightarrow C_{\text{sum}}$: Takes as input two commitments C, C' and outputs a homomorphically combined commitment C_{sum} .
- **HVC.OpenHom**(**pp**, Λ_i , Λ'_i) $\rightarrow \Lambda_{i,\text{sum}}$: Takes as input two opening proofs Λ_i, Λ'_i at the same index i , and outputs a homomorphically combined proof $\Lambda_{i,\text{sum}}$.

Security Definitions. An HVC scheme must satisfy position binding and statistical position hiding:

- **Position Binding:** For all PPT adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ produces a commitment C , index i , and two distinct messages $m \neq m'$ with valid proofs Λ, Λ' is negligible:

$$\Pr [m \neq m' \wedge \text{HVC.Ver}(\text{pp}, C, i, m, \Lambda) = 1 \wedge \text{HVC.Ver}(\text{pp}, C, i, m', \Lambda') = 1] \leq \text{negl}(\lambda).$$

- **Statistical Position Hiding:** For target index i^* and any two vectors $\vec{m}_0, \vec{m}_1$ that agree on all slots $j \neq i^*$, the distributions of $(C_0, \{\Lambda_{0,j}\}_{j \neq i^*})$ and $(C_1, \{\Lambda_{1,j}\}_{j \neq i^*})$ are statistically indistinguishable.

3.2 Concrete Construction

Let $\Omega = \{\omega^0, \dots, \omega^{n-1}\}$ be the n -th roots of unity, $Z(X) = X^n - 1$, and let $\mathbb{F}_p$ be the scalar field of BLS12-381.

HVC.Setup($1^\lambda, n$) $\rightarrow$ **pp**. Sample $\tau \leftarrow \mathbb{F}_p^*$ uniformly at random. Publish the degree- n KZG SRS as part of **pp**:

$$\text{SRS} = ([1]_1, [\tau]_1, \dots, [\tau^n]_1, [1]_2, [\tau]_2) \in \mathbb{G}_1^{n+1} \times \mathbb{G}_2^2.$$

Note that $[\tau^n]_2$ is not required because verification only involves $[\tau]_2 - [\omega^i]_2$. The hiding shift $[Z(\tau)]_1 = [\tau^n]_1 - [1]_1$ is obtained for free from the SRS. Precompute the table $W_j = [Z(\tau)/(\tau - \omega^j)]_1$ via a single $\mathbb{G}_1$ FFT of the SRS (data-independent, adding n compressed $\mathbb{G}_1$ elements). Public parameters are **pp** = (SRS, $\{W_j\}_{j=0}^{n-1}$).

At $n = 1024$, the verification SRS is $(n+1) \cdot 48 + 2 \cdot 96 + 48 = 48.3$ KB, which is $\approx 1070\times$ smaller than Catalano–Fiore (50.5 MB). Operational parameters, including the precomputed table $\{W_j\}$, total $48.3 + 48 = 96.3$ KB (a $512\times$ raw size reduction).

HVC.Com($\text{pp}, \vec{m}$) $\rightarrow (C, \text{aux})$. Interpolate $f(X) \in \mathbb{F}_p[X]$ of degree $< n$ such that $f(\omega^i) = m_i$ for all $i \in \{0, \dots, n-1\}$. Sample a random blinding factor $\rho \leftarrow \mathbb{F}_p$. Construct $F(X) = f(X) + \rho Z(X)$ and compute the commitment:

$$C = [F(\tau)]_1 = [f(\tau)]_1 + \rho[Z(\tau)]_1.$$

Output $(C, \text{aux} = \rho)$. The client retains ρ privately; the server never receives it.

HVC.Open($\text{pp}, \vec{m}, \text{aux}, i$) $\rightarrow \Lambda_i$. Compute the quotient polynomial $q_i(X) = (F(X) - m_i)/(X - \omega^i)$ and output the opening proof $\Lambda_i = [q_i(\tau)]_1$.

HVC.Ver($\text{pp}, C, i, m_i, \Lambda_i$) $\rightarrow 1/0$. Check if the following pairing equation holds:

$$e(C - [m_i]_1, [1]_2) = e(\Lambda_i, [\tau]_2 - [\omega^i]_2).$$

HVC.ComHom(pp, C, C') $\rightarrow C_{\text{sum}}$ & **HVC.OpenHom**($\text{pp}, \Lambda_i, \Lambda'_i$) $\rightarrow \Lambda_{i, \text{sum}}$. KHVC inherits additive homomorphism directly from the underlying KZG framework:

$$\text{HVC.ComHom}(\text{pp}, C, C') = C + C', \quad \text{HVC.OpenHom}(\text{pp}, \Lambda_i, \Lambda'_i) = \Lambda_i + \Lambda'_i.$$

Re-randomization can be executed via $\text{ReRand}(C) = C + \rho'[Z(\tau)]_1$, which updates the blinding scalar without altering any vector evaluations $F(\omega^i)$.

3.3 Formal Security Proofs

Theorem 1 (Position Binding). *If the q -SDH assumption holds with $q = n$, then KHVC satisfies position binding. An adversary with advantage ε yields a q -SDH solver with advantage $\varepsilon - O(1/p)$.*

Proof. Suppose a PPT adversary $\mathcal{A}$ outputs a commitment C , index i , and two distinct openings (m, Λ) and (m', Λ') for $m \neq m'$ such that both pass verification:

$$e(C - [m]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega^i]_2), \quad e(C - [m']_1, [1]_2) = e(\Lambda', [\tau]_2 - [\omega^i]_2).$$

Subtracting the two equations gives $e([m' - m]_1, [1]_2) = e(\Lambda - \Lambda', [\tau]_2 - [\omega^i]_2)$. By bilinear properties, this simplifies to:

$$[m' - m]_1 = (\tau - \omega^i)(\Lambda - \Lambda').$$

Since $m \neq m'$, $(m' - m)$ is invertible in $\mathbb{F}_p$ except with probability $O(1/p)$. Multiplying both sides by $(m' - m)^{-1}$ yields:

$$\left[\frac{1}{\tau - \omega^i} \right]_1 = (m' - m)^{-1}(\Lambda - \Lambda').$$

Given a q -SDH challenge instance $([1]_1, [\tau]_1, \dots, [\tau^n]_1, [1]_2, [\tau]_2)$, the reduction sets $c = -\omega^i$ and outputs $(c, [1/(\tau + c)]_1)$ as a valid solution to q -SDH with advantage $\varepsilon - O(1/p)$. $\square$

Theorem 2 (Statistical Position Hiding). *Let the adversary view C and $\{\Lambda_j\}_{j \neq i^*}$ as $\mathbb{G}_1$ elements. Except with probability at most n/p over setup (the event $\tau \in \Omega$), KHVC satisfies statistical position hiding with advantage at most n/p .*

Proof. The adversary chooses a target index i^* and two message vectors $\vec{m}_0, \vec{m}_1$ that agree on all slots $j \neq i^*$. Let $\Delta m = m_{i^*,1} - m_{i^*,0}$, and let $L_{i^*}(X)$ be the i^* -th Lagrange basis polynomial over Ω . The interpolated polynomials satisfy $f_1(X) = f_0(X) + \Delta m \cdot L_{i^*}(X)$.

Provided $\tau \notin \Omega$, $Z(\tau) \neq 0$. For any blinding factor $\rho_0 \in \mathbb{F}_p$ in world 0, define $\rho_1 = \rho_0 - \Delta m \cdot \frac{L_{i^*}(\tau)}{Z(\tau)}$. Evaluating $F_1(\tau)$ yields:

$$F_1(\tau) = f_1(\tau) + \rho_1 Z(\tau) = f_0(\tau) + \Delta m L_{i^*}(\tau) + \left(\rho_0 - \Delta m \frac{L_{i^*}(\tau)}{Z(\tau)} \right) Z(\tau) = F_0(\tau).$$

Hence, $C_1 = [F_1(\tau)]_1 = [F_0(\tau)]_1 = C_0$ is identically distributed in both worlds.

For any unrevealed opening index $j \neq i^*$, the opening element is $\Lambda_{b,j} = [(F_b(\tau) - m_{b,j})/(\tau - \omega^j)]_1$. Because $m_{0,j} = m_{1,j}$ for all $j \neq i^*$ and $F_0(\tau) = F_1(\tau)$, we have $\Lambda_{0,j} = \Lambda_{1,j}$.

Since $\rho_0 \mapsto \rho_1$ is a uniform bijection over $\mathbb{F}_p$, the joint distribution $(C_0, \{\Lambda_{0,j}\}_{j \neq i^*})$ and $(C_1, \{\Lambda_{1,j}\}_{j \neq i^*})$ seen by the adversary in $\text{Exp}_{\mathcal{A}, \text{HVC}}^{\text{PosHide}}$ are identical. Thus, an unbounded distinguisher has advantage at most n/p . $\square$

3.4 Efficiency Lemmas and Fast Updates

Lemma 1 (All-Openings Decomposition). *Let $F = f + \rho Z$ with $\deg f < n$ and $m_i = F(\omega^i) = f(\omega^i)$. Then:*

$$\frac{F(X) - m_i}{X - \omega^i} = \frac{f(X) - m_i}{X - \omega^i} + \rho \cdot \frac{Z(X)}{X - \omega^i}.$$

Equivalently, running FK20 [15] on the length- n prefix $(f_0 - \rho, f_1, \dots, f_{n-1})$ computes all baseline openings π_i^{FK20} , yielding $\Lambda_i = \pi_i^{\text{FK20}} + \rho \cdot W_i$, where $W_i = [Z(\tau)/(\tau - \omega^i)]_1$. KeyUp operations opening zero vectors reduce to n scalar multiplications over precomputed W_j , avoiding $O(n^2)$ naive MSMs.

Lemma 2 (One-Hot FileUp Openings). *Let $\Delta \in \mathbb{F}_p$, $i \in \{0, \dots, n-1\}$, and $f = \Delta L_i$ where $L_i(X) = \frac{\omega^i}{n} \frac{Z(X)}{X - \omega^i}$. Write $F = f + \rho Z$. For every $j \neq i$, the opening update is given in closed form by:*

$$\Lambda_j = \Delta \cdot \frac{\omega^i}{n} \cdot \frac{W_i - W_j}{\omega^i - \omega^j} + \rho \cdot W_j.$$

The opening at slot i is computed via a single length- n MSM. This avoids running dense FK20 algorithms when a single-slot vector update (FileUp) occurs.

4 Efficient IND-CPA Updatable Encryption

To handle large file payloads efficiently without relying on AES-GCM (which does not support direct ciphertext-independent re-encryption), we employ an efficient Data Encapsulation Mechanism (DEM) instantiated as a symmetric-key Updatable Encryption (UE) scheme. This construction is inspired by the Ring-LWR based updates used in CGLT23 [12] as well as the lightweight symmetric UE paradigm of SHINE/OCBSHINE [6] and ReCrypt [5].

4.1 Syntax and Security Definitions of UE

We use the ciphertext-independent sequential model of Lehmann and Tackmann [4], specialized to a symmetric DEM (no public key). Keys evolve by epochs $e = 0, 1, 2, \dots$; a single update token moves every ciphertext from epoch $e - 1$ to epoch e .

Syntax. A symmetric-key updatable encryption scheme UE consists of six algorithms:

- $\text{UE.Setup}(1^\lambda) \rightarrow \text{pp}$: outputs public parameters.
- $\text{UE.KeyGen}(\text{pp}) \rightarrow s_0$: samples the initial epoch secret key.
- $\text{UE.NextKey}(s_e) \rightarrow (s_{e+1}, \Delta_{e+1})$: samples the next epoch key and a ciphertext-independent update token.
- $\text{UE.Enc}(s_e, \text{tweak}, m) \rightarrow \text{ct}$: encrypts m under the current key (and an optional public tweak).
- $\text{UE.Dec}(s_e, \text{tweak}, \text{ct}) \rightarrow m/\perp$: decrypts.
- $\text{UE.ReEnc}(\Delta_{e+1}, \text{ct}) \rightarrow \text{ct}'$: updates a ciphertext from epoch e to epoch $e+1$ using only the token.

Correctness requires $\text{UE.Dec}(s_e, \text{tweak}, \text{UE.Enc}(s_e, \text{tweak}, m)) = m$ and, after any sequence of honest updates of length at most the scheme's noise budget, $\text{UE.Dec}(s_{e+t}, \text{tweak}, \text{UE.ReEnc}^*(\text{UE.Enc}(s_e, \text{tweak}, m))) = m$.

Oracles (following [4, §3.2]). In the security experiments the challenger maintains an epoch counter e , the current key s_e , the current token Δ_e , a challenge epoch $\tilde{e}$ (initially $\perp$), and lists

- $\mathcal{L}$: honest non-challenge ciphertexts (c, e) produced by $\mathcal{O}^{\text{Enc}}$ or $\mathcal{O}^{\text{ReEnc}}$;
- $\tilde{\mathcal{L}}$: updated versions of the challenge ciphertext;
- $\mathcal{C}$: epochs in which the adversary obtained a challenge ciphertext (via $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$);
- $\mathcal{K}$: epochs in which the adversary corrupted a secret key;
- $\mathcal{T}$: epochs in which the adversary corrupted an update token.

The adversary may query:

$\mathcal{O}^{\text{Enc}}(m)$
return $c \leftarrow \text{UE.Enc}(s_e, \text{tweak}, m)$, add (c, e) to $\mathcal{L}$.

$\mathcal{O}^{\text{Next}}$
run $(s_{e+1}, \Delta_{e+1}) \leftarrow \text{UE.NextKey}(s_e)$ and set $e \leftarrow e+1$; if the challenge has already been issued, also update the challenge ciphertext $\tilde{c}_e \leftarrow \text{UE.ReEnc}(\Delta_e, \tilde{c}_{e-1})$ and store it in $\tilde{\mathcal{L}}$.

$\mathcal{O}^{\text{ReEnc}}(c)$
require $(c, e-1) \in \mathcal{L}$; return $c' \leftarrow \text{UE.ReEnc}(\Delta_e, c)$ and add (c', e) to $\mathcal{L}$.

$\mathcal{O}^{\text{Corrupt}}(\text{key}, e^*) / \mathcal{O}^{\text{Corrupt}}(\text{token}, e^*)$
for $e^* \leq e$, return s_{e^*} (add e^* to $\mathcal{K}$) or Δ_{e^*} (add e^* to $\mathcal{T}$), respectively.

$\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$
return the current challenge ciphertext $\tilde{c}_e \in \tilde{\mathcal{L}}$ and add e to $\mathcal{C}$.

Extended sets $\mathcal{C}^*, \mathcal{K}^*, \mathcal{T}^*$ close $\mathcal{C}, \mathcal{K}, \mathcal{T}$ under the scheme's leakage profile (token inference and uni-/bi-directional updates) as in [4, §3.3]. A trivial win occurs if $\mathcal{C}^* \cap \mathcal{K}^* \neq \emptyset$ (the adversary knows a challenge ciphertext and the matching epoch key).

Definition 1 (IND-ENC-CPA [4]). An updatable encryption scheme UE is IND-ENC-CPA secure if for every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{Exp}_{\mathcal{A}, \text{UE}}^{\text{IND-ENC-CPA}}(\lambda) = 1] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment proceeds as follows.

1. The challenger samples $s_0 \leftarrow \text{UE.KeyGen}(\text{UE.Setup}(1^\lambda))$, sets $e \leftarrow 0$, $\tilde{e} \leftarrow \perp$, and initializes empty lists $\mathcal{L}, \tilde{\mathcal{L}}, \mathcal{C}, \mathcal{K}, \mathcal{T}$.
2. $\mathcal{A}$ may adaptively query $\mathcal{O}^{\text{Enc}}$, $\mathcal{O}^{\text{Next}}$, $\mathcal{O}^{\text{ReEnc}}$, and $\mathcal{O}^{\text{Corrupt}}$.
3. $\mathcal{A}$ outputs a challenge (m_0, m_1, st) with $|m_0| = |m_1|$. The challenger samples $b \leftarrow \{0, 1\}$, sets $\tilde{e} \leftarrow e$, $\tilde{c} \leftarrow \text{UE.Enc}(s_{\tilde{e}}, \text{tweak}, m_b)$, stores $(\tilde{c}, \tilde{e})$ in $\tilde{\mathcal{L}}$, and returns $\tilde{c}$ to $\mathcal{A}$.
4. $\mathcal{A}$ continues with the same oracles plus $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$, then outputs a bit b' .
5. The experiment returns 1 iff $b' = b$ and $\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$ (no secret-key corruption in any challenge-equal epoch).

This is the standard CPA game lifted to sequential key rotation with adaptive key/token corruption and post-compromise security [4]: fresh encryptions hide the plaintext even if keys before/after the challenge epoch leak, provided the adversary never obtains both a challenge ciphertext and its epoch key.

Definition 2 (IND-UPD-CPA [4]). An updatable encryption scheme UE is IND-UPD-CPA secure if for every PPT adversary $\mathcal{A}$,

$$\left| \Pr[\text{Exp}_{\mathcal{A}, \text{UE}}^{\text{IND-UPD-CPA}}(\lambda) = 1] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment proceeds as follows.

1. Setup is identical to Definition 1.
2. $\mathcal{A}$ queries $\mathcal{O}^{\text{Enc}}$, $\mathcal{O}^{\text{Next}}$, $\mathcal{O}^{\text{ReEnc}}$, and $\mathcal{O}^{\text{Corrupt}}$.
3. $\mathcal{A}$ outputs two ciphertexts (c_0, c_1, st) that both appear in $\mathcal{L}$ at epoch $e - 1$ and satisfy $|c_0| = |c_1|$. The challenger samples $b \leftarrow \{0, 1\}$, sets $\tilde{e} \leftarrow e$, $\tilde{c} \leftarrow \text{UE.ReEnc}(\Delta_{\tilde{e}}, c_b)$, stores $(\tilde{c}, \tilde{e})$ in $\tilde{\mathcal{L}}$, and returns $\tilde{c}$ to $\mathcal{A}$.
4. $\mathcal{A}$ continues with the same oracles plus $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$, then outputs b' .
5. The experiment returns 1 iff $b' = b$ and all of the following hold:
 - (a) $\tilde{e} \notin \mathcal{T}^*$ (the challenge-epoch token was not corrupted);
 - (b) $\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$ (no key corruption in a challenge-equal epoch);
 - (c) if UE.ReEnc is deterministic, then $\mathcal{A}$ did not query $\mathcal{O}^{\text{ReEnc}}(c_0)$ or $\mathcal{O}^{\text{ReEnc}}(c_1)$ in epoch $\tilde{e}$.

IND-UPD-CPA is the unlinkability game of [4]: an updated ciphertext hides which of two equal-length prior ciphertexts it came from, even under adaptive corruptions outside the trivial-win region. (Our Ring-LWR UE.ReEnc is deterministic, so condition (c) is enforced by the experiment rather than being vacuous. RISE updates are randomized, for which (c) would be vacuous.)

Remark 1 (DEM specialization used in USS). In the USS hybrids of Section 5 the body UE is only accessed as a CPA DEM: the reduction never corrupts the challenge-epoch DEM secret, and KeyUp is the honest $\text{NextKey}/\text{ReEnc}$ path. Remark 2 is exactly the IND-ENC-CPA hop (uniform bodies) and the IND-UPD-CPA hop (updated bodies $\approx$ fresh encryptions under s') under that restriction. Full adaptive post-compromise security of the Ring-LWR DEM as a standalone UE is Theorem 3 (citing [9, 8, 7]), with at most 127 updates inside the noise pad.

4.2 Concrete Construction from Ring-LWR

Let $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and $R_p = \mathbb{Z}_p[X]/(X^N + 1)$, where $N = 1024$, $q = 998244353$ is a 30-bit NTT-friendly prime, and $p = 2^{16}$ is the Ring-LWR rounding modulus (distinct from the BLS12-381 scalar field $\mathbb{F}_p$ used elsewhere; cf. Section 2). Let $H : \{0, 1\}^* \rightarrow R_q$ be a hash function modeled as a Random Oracle.

UE.Setup(1^λ). Output $\text{pp} = (N, q, p, H)$.

UE.KeyGen(pp). Sample a uniform secret $s \leftarrow R_q$.

UE.Enc(s, tweak, m). For each block index b of message $m \in R_p$:

1. Compute $a_b = H(\text{tweak} \parallel b) \in R_q$.
2. Compute pseudo-random pad $F_s(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor \in R_p$.
3. Output ciphertext block $c_b = m_b + F_s(b) \pmod{p}$.

In implementation, a_b is computed in the twisted NTT domain and cached per file block alongside tweak .

UE.Dec(s, tweak, c_b). Recompute $a_b = H(\text{tweak} \parallel b)$, evaluate $F_s(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor \in R_p$, and recover $m_b = c_b - F_s(b) \pmod{p}$.

UE.ReEnc(Δ, c_b). Given the key transition token $\Delta = s' - s \in R_q$ and cached NTT of a_b :

1. Evaluate $F_\Delta(b) = \left\lfloor \frac{p}{q}(a_b \cdot \Delta) \right\rfloor \in R_p$.
2. Homomorphically update the ciphertext: $c'_b = c_b + F_\Delta(b) \pmod{p}$.

Noise Accumulation and Decryption Bound. Updating directly in $\mathbb{Z}_p$ introduces a rounding error of infinity norm $\|e\|_\infty \leq 1$ per rotation turn. After t consecutive updates, $\|e^{(t)}\|_\infty \leq t$. By reserving noise bits in each coefficient encoding, decryption is exact for up to $t \leq 127$ updates.

Theorem 3 (Ring-LWR DEM confidentiality). *Assume decisional Ring-LWR on R_q with rounding modulus p is hard, and model H as a random oracle. Let UE be the scheme above, restricted to at most $t \leq 127$ honest updates (so the noise pad never overflows). Then:*

1. $F_s(b) = \left\lfloor (p/q)(a_b \cdot s) \right\rfloor$ is a secure almost key-homomorphic PRF in the ROM [9, 8] (as recalled for UE in [7, §6]).
2. UE is IND-ENC-CPA secure (Definition 1).
3. UE is IND-UPD-CPA secure (Definition 2).

In particular, a fresh body is indistinguishable from uniform in R_p , and an honestly updated body is indistinguishable from a fresh encryption under the new secret s' (up to a message-independent rounding error of infinity norm at most t).

Proof. PRF security (prior work). For each block index b , the public ring element $a_b = H(\text{tweak} \| b)$ is a uniformly random oracle output. The map $s \mapsto \lfloor (p/q)(a_b \cdot s) \rfloor$ is the standard Ring-LWR sample. Banerjee–Peikert–Rosen [9] and Boneh–Lewi–Montgomery–Raghunathan [8] prove that the keyed function $F_s(x) = \lfloor (p/q)(H(x) \cdot s) \rfloor$ is a secure PRF (and γ -almost key-homomorphic with $\gamma = 1$ for a single sample) in the random-oracle model under decisional (Ring-)LWR. Boneh–Eskandarian–Kim–Fisch [7, §6] explicitly recall this Ring-LWR almost KH-PRF as the lattice baseline for updatable encryption (and then replace it by RLWE only to enlarge the noise budget). We inherit that PRF theorem unchanged; our parameters merely fix a concrete 8-bit noise pad of capacity 127.

Almost key-homomorphism. For any $s, s' \in R_q$ and $\Delta = s' - s$,

$$F_s(b) + F_\Delta(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor + \left\lfloor \frac{p}{q}(a_b \cdot \Delta) \right\rfloor = F_{s'}(b) + e_b$$

with a coefficient-wise rounding error $e_b \in \{-1, 0, 1\}^N$. Hence t successive updates accumulate $\|e\|_\infty \leq t$, which the 8-bit noise pad absorbs for $t \leq 127$.

IND-ENC-CPA. This is the standard “counter-mode from a PRF” argument, in the sequential UE game of Definition 1 (cf. the KH-PRF UE analyses of [8, 5] and the confidentiality reduction for [7, Construction 5.2]). Fix a PPT adversary $\mathcal{A}$. As long as $\mathcal{A}$ does not trigger a trivial win ($\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$), every challenge-equal epoch key is unknown to $\mathcal{A}$.

- *Hybrid H_0 :* the real IND-ENC-CPA experiment.
- *Hybrid H_1 :* replace every evaluation of $F_{s_{\tilde{e}}}(\cdot)$ used to form the challenge ciphertext (and its later updates via $\mathcal{O}^{\text{upd}\tilde{\mathcal{C}}}$, which only add public F_Δ -pads once Δ is known) by independent uniform elements of R_p .

Distinguishing H_0 from H_1 is exactly the PRF advantage against F under the unknown challenge-epoch secret (or, after an update into a new uncorrupted epoch, under that epoch’s secret), which is negligible by decisional Ring-LWR in the ROM. In H_1 the challenge ciphertext is m_b plus uniform pads, hence independent of the hidden bit b . Thus $\text{Adv}^{\text{IND-ENC-CPA}} \leq \text{Adv}_F^{\text{PRF}} \leq \text{Adv}^{\text{RLWR}}$.

IND-UPD-CPA. Again follow the almost-KH-PRF UE template [5, 7]. In Definition 2 the challenge is $\tilde{c} = \text{UE.ReEnc}(\Delta_{\tilde{e}}, c_b)$ for a hidden bit b , and $\tilde{e} \notin \mathcal{T}^*$ (the challenge-epoch token is not corrupted).

- *Hybrid G_0 :* the real experiment.
- *Hybrid G_1 :* write the challenge as $\text{encode}(m_b) + F_{s_{\tilde{e}}}(\cdot) + e$ for a message-independent rounding error $\|e\|_\infty \leq 1$, where m_b is the (adversary-unknown) plaintext underlying c_b . Replace $F_{s_{\tilde{e}}}(\cdot)$ by a uniform pad $U \in R_p$ (PRF / Ring-LWR under the unknown key $s_{\tilde{e}}$).
- *Hybrid G_2 :* the challenge is now $\text{encode}(m_b) + U + e$. Since U is uniform in R_p , so is the challenge, independently of the hidden bit b (and of which prior ciphertext was updated).

The hop $G_0 \rightarrow G_1$ is Ring-LWR; G_1 and G_2 are identically distributed. Hence $\text{Adv}^{\text{IND-UPD-CPA}} \leq \text{Adv}^{\text{RLWR}}$. (The same hop shows that an updated body is indistinguishable from a fresh encryption under s' , up to the absorbed rounding error—the specialization used in Remark 2.)

What is not new. We do not claim a new lattice assumption or a new KH-PRF. The PRF theorem is [9, 8]; the reduction from almost KH-PRF counter-mode to UE confidentiality is the standard one used by [8, 5, 7]. What this paper contributes is the *instantiation and measurement* of the Ring-LWR DEM inside USS under a 127-update pad (whereas [7] abandoned Ring-LWR for RLWE to target a larger update budget). $\square$

5 Updatable Secure Storage (USS)

In this section, we present the formal syntax and security definitions for Updatable Secure Storage (USS), followed by our concrete construction using KHVC, RISE, and our Ring-LWR UE scheme, and complete security proofs.

5.1 USS Syntax and Security Definitions

Syntax. An Updatable Secure Storage (USS) scheme consists of six algorithms:

- $\text{USS.ParGen}(1^\lambda, n) \rightarrow \text{pp}$: Takes as input the security parameter 1^λ and maximum capacity n , outputting public parameters pp .
- $\text{USS.KeyGen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$: Outputs client secret key sk and public key pk .
- $\text{USS.Store}(\text{pp}, \text{sk}, \vec{m}) \rightarrow (\text{stub}, \text{repo})$: Encrypts dataset $\vec{m} = (m_1, \dots, m_n)$, returning a local client stub stub and server repository repo .
- $\text{USS.Rev}(\text{pp}, \text{sk}, \text{repo}, i) \rightarrow m_i / \perp$: Decrypts and verifies slot i , returning m_i or failure symbol $\perp$.
- $\text{USS.FileUp}(\text{pp}, \text{sk}, \text{repo}, i, m'_i) \rightarrow (\text{stub}', \text{repo}')$: Updates slot i to m'_i with communication $O(1) + |m'_i|$ (independent of n): the client sends $(i, \Delta h_i, \rho_\delta, C_\delta)$ plus a fresh UE body, and the server recomputes/RISE-encrypts the one-hot openings under the public key Y .
- $\text{USS.KeyUp}(\text{pp}, \text{sk}, \text{sk}', \text{repo}) \rightarrow (\text{token}, \text{repo}')$: Generates token token allowing server-side key rotation from sk to sk' .

Security Definitions. We adopt the four USS games of CGLT23 [12, §3.2] verbatim (only notation aligned with our algorithms). A challenge ciphertext is the encryption of a target file; after key rotation its image is called a *challenge-equal* ciphertext, and the repository containing them is the challenge-equal repository.

Challenger state [12]. The challenger maintains: e (current epoch, init. 1); e^* (challenge epoch, init. $\perp$); sb^* (stub of the challenge-equal repository, init. $\perp$); $\mathcal{K}$ (epochs where the adversary corrupted the epoch key); $\mathcal{C}$ (epochs where it corrupted a challenge-equal ciphertext); $\mathcal{I}$ (indices of challenge-equal ciphertexts; empty before the challenge); $\mathcal{L}$ (tuples $(t, \text{sb}, \text{repo}, \text{db})$ tracking all repositories); $\mathcal{S}$ (tuples (t, Trans) of key-update transcripts from epoch $t - 1$ to t); $\mathcal{F}$ (tuples $(\text{sb}, t, i, \text{ft}_i)$ of file-update transcripts on challenge-equal slots); $\mathcal{T}$ (epochs where the adversary queried a key-update transcript).

Confidentiality oracles [12].

$\mathcal{O}.\text{Store}(\text{db})$

run $(\text{sb}, \text{repo}) \leftarrow \text{Store}(\text{sk}_e, \text{db}, \text{pp})$, return (sb, repo) , add $(e, \text{sb}, \text{repo}, \text{db})$ to $\mathcal{L}$.

$\mathcal{O}.\text{Next}$

for every $(e, \text{sb}, \text{repo}, \text{db}) \in \mathcal{L}$ sample $\text{sk}' \leftarrow \text{KeyGen}(\text{pp})$ and run KeyUp to obtain $(\text{sb}', \text{repo}', \text{Trans})$; set $e \leftarrow e + 1$, append updated entries to $\mathcal{L}$ and (e, Trans) to $\mathcal{S}$; if $\text{sb} = \text{sb}^*$ then update $\text{sb}^* \leftarrow \text{sb}'$; return to $\mathcal{A}$ all updated $(\text{sb}', \text{repo}')$ for non-challenge repositories, and for the challenge-equal repository only the stub and ciphertexts with indices $j \notin \mathcal{I}$.

$\mathcal{O}.\text{KeyCorr}(t)$

if $t \leq e$, return sk_t and add t to $\mathcal{K}$.

$\mathcal{O}.\text{KeyUpTrans}(t)$

if $t \leq e$, return the transcript Trans of the update from epoch $t - 1$ to t and add t to $\mathcal{T}$.

$\mathcal{O}.\text{FileUp}(\text{sb}, m'_i, i)$

retrieve $(e, \text{sb}, \text{repo}, \text{db}) \in \mathcal{L}$; run FileUp to $(\text{sb}', \text{repo}')$; if $\text{sb} \neq \text{sb}^*$, return $(\text{sb}', \text{repo}', \text{ft}_i)$; if $\text{sb} = \text{sb}^*$, set $\text{sb}^* \leftarrow \text{sb}'$; if $i \in \mathcal{I}$ remove i from $\mathcal{I}$ and record $(\text{sb}, e, i, \text{ft}_i)$ in $\mathcal{F}$; return sb' and all updated ciphertexts with indices $j \notin \mathcal{I}$ (and ft_i if $i \notin \mathcal{I}$).

$\mathcal{O}.\text{ChaCTCorr}(j)$

if $j \in \mathcal{I}$ and $\text{sb}^* \neq \perp$, add e to $\mathcal{C}$ and return the j -th ciphertext of the challenge-equal repository; else $\perp$.

$\mathcal{O}.\text{ChaFTCorr}(\text{sb}, t, i)$

return the recorded challenge-equal file-update transcript ft_i if present.

Trivial win [12]. Key-update transcripts may update or downgrade challenge-equal ciphertexts (bi-directional leakage of our building blocks). Define the extended corruption set $\tilde{\mathcal{C}}$ by $i \in \tilde{\mathcal{C}}$ iff $i \in \mathcal{C}$, or $(i - 1 \in \mathcal{C} \wedge i \in \mathcal{T})$, or $(i + 1 \in \mathcal{C} \wedge i + 1 \in \mathcal{T})$. Any confidentiality experiment that ends with $\mathcal{K} \cap \tilde{\mathcal{C}} \neq \emptyset$ is a trivial win and returns $\perp$ (advantage ignored).

Definition 3 (IND-DD-UP [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, b)$:

1. $\text{pp} \leftarrow \text{ParGen}(1^\lambda, \mathcal{M}, n)$; initialize $e = 1$, $e^* = \text{sb}^* = \perp$ and empty $\mathcal{K}, \mathcal{C}, \mathcal{I}, \mathcal{L}, \mathcal{S}, \mathcal{T}, \mathcal{F}$; $\text{sk}_1 \leftarrow \text{KeyGen}(\text{pp})$.
2. $(\text{db}_0, \text{db}_1, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$ with $\text{db}_b = (m_{b,1}, \dots, m_{b,n})$.
3. $(\text{sb}^*, \text{repo}^*) \leftarrow \text{Store}(\text{db}_b, \text{sk}_e, \text{pp})$; set $e^* \leftarrow e$; add $(e, \text{sb}^*, \text{repo}^*, \text{db}_b)$ to $\mathcal{L}$; for each i with $m_{0,i} \neq m_{1,i}$ add i to $\mathcal{I}$.
4. $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}, \mathcal{O}.\text{ChaFTCorr}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{repo}^*)$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-DD-UP secure if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}} := |\Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, 0) = 1] - \Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, 1) = 1]|$$

is negligible. Intuitively: even with adaptive key/file/key-update corruptions (outside trivial wins), $\mathcal{A}$ cannot tell which of two equal-length databases was stored.

Definition 4 (IND-FileUp-CPA [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-FileUp-CPA}}(1^\lambda, \mathcal{M}, n, b)$:

1. Setup as in Definition 3.
2. $(\text{sb}, i, m_{0,i}, m_{1,i}, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$. Abort if $(e, \text{sb}, \cdot, \cdot) \notin \mathcal{L}$, or $i \notin [1, n]$, or $|m_{0,i}| \neq |m_{1,i}|$.
3. Retrieve $(e, \text{sb}, \text{repo}, \text{db})$; set $e^* \leftarrow e$, $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$, $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$. Run FileUp of slot i to $m_{b,i}$, obtaining $(\text{sb}^*, \text{repo}^*)$ and transcript ft ; add $(e, \text{sb}^*, \text{repo}^*, \text{db}_b)$ to $\mathcal{L}$ where db_b replaces coordinate i by $m_{b,i}$.
4. $b' \leftarrow \mathcal{A}^{\dots, \mathcal{O}.\text{ChaFTCorr}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{sb}^*, \text{repo}^*, \text{ft})$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-FileUp-CPA secure if the analogous distinguishing advantage is negligible. Intuitively: a FileUp hides which of two equal-length replacement files was written, even if $\mathcal{A}$ knew the previous database.

Definition 5 (IND-REENC-CPA [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-REENC-CPA}}(1^\lambda, \mathcal{M}, n, b)$:

1. Setup as in Definition 3 (USS epoch counter starts at $e = 1$).
2. $(\text{sb}_0, \text{sb}_1, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$.
3. Retrieve $(\text{sb}_0, \text{repo}_0, \text{db}_0)$ and $(\text{sb}_1, \text{repo}_1, \text{db}_1)$ from $\mathcal{L}$ at the current epoch; set $\mathcal{I} = \{i : c_{0,i} \neq c_{1,i}\}$, sample a fresh epoch key, set $e \leftarrow e + 1$, $e^* \leftarrow e$, $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$. Run *KeyUp* on repository b to obtain $(\text{sb}^*, \text{repo}^*)$; honestly *KeyUp* every other repository as well.
4. $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{sb}^*, \text{repo}^*, \{\text{all other updated } (\text{sb}', \text{repo}')\})$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-REENC-CPA secure if the distinguishing advantage is negligible. Intuitively: after *KeyUp*, $\mathcal{A}$ cannot tell which of two previously stored repositories was rotated (unlinkability / post-compromise security for the whole repository), even with *FileUp* oracles.

Definition 6 (OF-PTXT [12]). The integrity challenger keeps a list $\mathcal{R}$ of the latest honest pairs (sb, db) (only the current epoch). The adversary (malicious server) gets:

$\mathcal{O}.\text{StoreINT}(\text{db})$

run *Store*, return (sb, repo) , add (sb, db) to $\mathcal{R}$.

$\mathcal{O}.\text{NextINT}$

advance the epoch, sample a new key, run the client side of *KeyUp* on every stub, return all new stubs and client transcripts Trans_e ; update every $(\text{sb}, \text{db}) \in \mathcal{R}$ to (sb', db) (plaintexts unchanged).

$\mathcal{O}.\text{FileUpINT}(m'_i, i, \text{sb})$

if $\text{sb} \in \mathcal{R}$, run the client *FileUp* protocol against the adversarial server (who may deviate arbitrarily); if the client accepts, update $\mathcal{R}$ to (sb', db') .

$\mathcal{O}.\text{KeyCorr}$

return the current epoch secret (integrity must survive key leakage).

Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{OF-PTXT}}(1^\lambda, \mathcal{M}, n)$:

1. $\text{pp} \leftarrow \text{ParGen}$; initialize $\mathcal{R}$, e ; $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$.
2. $(\text{sb}, i, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{StoreINT}, \mathcal{O}.\text{NextINT}, \mathcal{O}.\text{FileUpINT}, \mathcal{O}.\text{KeyCorr}}(\text{pp})$. Abort if $(\text{sb}, \cdot) \notin \mathcal{R}$ or $i \notin [1, n]$.
3. Run *Rev* for slot i against $\mathcal{A}$ (adversarial repository) and obtain m_i^* .
4. Return 1 iff for every $(\text{sb}, \text{db}) \in \mathcal{R}$ one has $m_i^* \neq \text{db}[i]$ (and *Rev* accepted); else 0.

USS is OF-PTXT secure if $\Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{OF-PTXT}} = 1]$ is negligible for every PPT $\mathcal{A}$. Intuitively: even a malicious server that knows the current secret and may refuse to follow *FileUp*/*KeyUp* cannot make the client accept a plaintext that is not the latest honest version bound to the client's stub.

5.2 Concrete Construction

We build USS using KHVC (Section 3), RISE in $\mathbb{G}_1$, and the Ring-LWR UE scheme (Section 4). Let $\text{COM}(m_i; \nu_i) = \text{SHA-256}(m_i \parallel \nu_i)$ mapped into $\mathbb{F}_p$ as a little-endian scalar. In the ROM, COM is hiding (256-bit random nonce ν_i) and binding.

$\text{USS.ParGen}(1^\lambda, n)$. Run $\text{KHVC.Setup}(1^\lambda, n)$ to generate $\text{pp}_{\text{HVC}} = (\text{SRS}, \{W_j\}_{j=0}^{n-1})$. Output $\text{pp} = \text{pp}_{\text{HVC}}$.

USS.KeyGen(pp). Sample RISE key pair $(k, Y = kg_1)$ with $g_1 \in \mathbb{G}_1$ and Ring-LWR secret $s \in R_q$. Output $\text{sk} = (k, s)$ and $\text{pk} = Y$.

USS.Store(pp, sk, $(m_1, \dots, m_n)$). For each $i \in \{1, \dots, n\}$:

1. Sample 256-bit nonce ν_i and fresh tweak_i . Compute commitment $h_i = \text{COM}(m_i; \nu_i) \in \mathbb{F}_p$.
2. Encrypt payload $\nu_i \parallel m_i$ as $C_i^{\text{UE}} \leftarrow \text{UE.Enc}(s, \text{tweak}_i, \nu_i \parallel m_i)$ and cache a_b NTTs.
3. Compute KHVC opening proof Λ_i for $(h_1, \dots, h_n)$, and RISE-encrypt it: $E_i = \text{RISE.Enc}_k(\Lambda_i)$.

Sample fresh scalar $\rho \in \mathbb{F}_p$. Compute $(C, \rho) \leftarrow \text{KHVC.Com}(\text{pp}, (h_1, \dots, h_n))$. The client keeps local stub $\text{stub} = C$, discards ρ , and sends repository $\text{repo} = (\{E_i\}, \{C_i^{\text{UE}}\}, \{\text{tweak}_i\})$ to the server.

USS.Rev(pp, sk, repo, i). The client downloads E_i, C_i^{UE} , and tweak_i . It decrypts $\Lambda_i \leftarrow \text{RISE.Dec}_k(E_i)$, decrypts payload $(\nu_i, m_i) \leftarrow \text{UE.Dec}(s, \text{tweak}_i, C_i^{\text{UE}})$, and recomputes $h_i = \text{COM}(m_i; \nu_i)$. It returns m_i if $\text{KHVC.Ver}(\text{pp}, C, i, h_i, \Lambda_i) = 1$, else $\perp$.

USS.FileUp(pp, sk, repo, i, m'_i). The client downloads slot i , extracts (m_i, ν_i) , samples fresh ν'_i , and computes $h'_i = \text{COM}(m'_i; \nu'_i)$. It sets $\Delta h_i = h'_i - h_i$, samples $\rho_\delta \in \mathbb{F}_p$, and forms $C_\delta = [\Delta h_i L_{i-1}(\tau)]_1 + \rho_\delta [Z(\tau)]_1$ (identifying USS slot i with KHVC index $i - 1$). It sends the update token

$$(i, \Delta h_i, \rho_\delta, C_\delta, (C_i^{\text{UE}})', \text{tweak}'_i)$$

of size $O(1) + |m'_i|$ (independent of n) to the server. The server:

1. checks that C_δ equals the public commitment of the one-hot delta $(\Delta h_i, \rho_\delta)$ at index $i - 1$;
2. updates the stub by $\text{stub}' = \text{stub} + C_\delta$;
3. recomputes the one-hot openings $\{\Lambda_{\delta,j}\}_j$ via Lemma 2 from the public SRS and $(\Delta h_i, \rho_\delta)$;
4. freshly RISE-encrypts each $\Lambda_{\delta,j}$ under the public key Y and additively updates $E_j \leftarrow E_j + \text{RISE.Enc}_Y(\Lambda_{\delta,j})$;
5. replaces the body of slot i by the fresh UE ciphertext.

(The values $\Delta h_i, \rho_\delta$ are sent in the clear so that the server—who holds only Y , not k —can produce the RISE envelopes locally; revealing the field difference of two hash commitments does not open the payloads, which remain protected by COM and the UE body.)

USS.KeyUp(pp, sk, sk', repo). Let $\text{sk} = (k, s)$ and $\text{sk}' = (k', s')$. The client sends token $\text{token} = (\Delta_R = k'/k, Y' = k'g_1, \rho_0 \in \mathbb{F}_p, \Delta = s' - s)$. The server:

1. Updates RISE headers using Δ_R and adds $\text{RISE.Enc}_{k'}(\rho_0 \cdot W_j)$ to each opening.
2. Updates local stub: $\text{stub}' = \text{stub} + \rho_0 [Z(\tau)]_1$.
3. Re-encrypts payload bodies via $\text{UE.ReEnc}(\Delta, C_i^{\text{UE}})$ using cached a_b NTTs.

5.3 Security Proofs

We prove confidentiality by hybrid arguments aligned with [12, Thms. 3–5], under the substitutions of Section 2.4: RISE in $\mathbb{G}_1$ (SXDH), Type-3 pairing checks, KHVC in place of Catalano–Fiore, and the Ring-LWR UE of Section 4 as the DEM. Integrity (OF-PTXT) is Lemma 6.

Theorem 4 (USS Security). *Under the hypotheses of Section 2.4, and writing Adv for the USS advantages of [12]:*

1. KHVC is statistically position hiding (Theorem 2) and tightly q -SDH position binding.
2. IND-DD-UP, IND-FileUp-CPA, and IND-REENC-CPA hold as proved below.
3. OF-PTXT holds by Lemma 6, even if the current RISE key, DEM secret, and every KeyUp nonce ρ_0 leak.
4. A fresh body (and a body after at most 127 updates) is IND-CPA under decisional Ring-LWR in the ROM. The DEM is not authenticated encryption.

Remark 2 (DEM in the random oracle). *Let H be the random oracle used as COM and as $a_b = H(\text{tweak} \| b)$. A body block is of the form $\text{encode}(\nu \| m) + F_s(b)$ with $F_s(b) = \lfloor (p/q)(a_b \cdot s) \rfloor \in R_p$. Replacing every block of a challenge body by an independent uniform element of R_p is a standard decisional Ring-LWR hop for the epoch secret s : each a_b is a fresh oracle string (fresh tweak, or a previously unused block index), so the rounded products are Ring-LWR samples. The same hop applies after KeyUp. The server receives $\Delta = s' - s$. Since s' is uniform in R_q , Δ is uniform and independent of s , and an honestly updated block equals*

$$\text{encode}(\nu \| m) + F_{s'}(b) + e \quad \text{with} \quad \|e\|_\infty \leq t$$

after t updates. For $t \leq 127$ the noise pad removes e on decryption, and e is independent of the message bits. Distinguishing the updated body from uniform R_p is therefore Ring-LWR under s' . Seeing both the pre- and post-update bodies does not help beyond evaluating F_Δ , which the server can already compute from the public token Δ and the public a_b .

Lemma 3 (IND-DD-UP). *Any PPT adversary $\mathcal{A}$ against IND-DD-UP of our USS satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-DD-UP}} \leq |\mathcal{I}| \cdot \left(\text{Adv}_{\mathcal{A}}^{\text{RO-guess}} + \text{Adv}_{\mathcal{B}_1}^{\text{RISE-ENC}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p} \right),$$

where $\mathcal{I}$ is the set of differing challenge slots from Definition 3 (so $|\mathcal{I}| \leq n$), the first term inside the parentheses is the probability that $\mathcal{A}$ queries H on a challenge nonce before it is revealed, $\mathcal{B}_1$ is a RISE IND-ENC adversary under SXDH, and $\mathcal{B}_2$ is a decisional Ring-LWR adversary in the ROM.

Proof. We follow the hybrid outline of [12, Thm. 3]. Let $\mathcal{I} = \{i : m_{0,i} \neq m_{1,i}\}$ be the set of differing slots from Definition 3, and write $|\mathcal{I}| \leq n$. We change the challenge database from db_0 to db_1 one slot at a time: for $\ell = 0, \dots, |\mathcal{I}|$, hybrid $H^{(\ell)}$ stores db_1 on the first ℓ indices of $\mathcal{I}$ (in any fixed order) and db_0 on the rest. It suffices to bound the advantage of distinguishing $H^{(\ell)}$ from $H^{(\ell+1)}$ for a single challenge slot $i^* \in \mathcal{I}$; multiplying by $|\mathcal{I}|$ yields the lemma statement. Fix such an i^* and equal-length challenge plaintexts (m_0^*, m_1^*) , and write H_b for the slot-wise hybrids parameterized by the hidden bit $b \in \{0, 1\}$.

Game H_0 (real). The challenger runs ParGen/KeyGen, stores the challenge vector with $m_{i^*} = m_b^*$, and answers FileUp/KeyUp/Rev honestly. $\mathcal{A}$ sees the full repository (RISE openings, Ring-LWR bodies, tweaks) and the stub C .

Game H_1 (hide COM). Identical to H_0 , except the challenger samples $h_{i^*} \leftarrow \mathbb{F}_p$ uniformly instead of $h_{i^*} = \text{COM}(m_b^*; \nu_{i^*})$, and programs the random oracle so that $H(m_b^* \| \nu_{i^*}) = h_{i^*}$ only if

$\mathcal{A}$ never queries that point. If $\mathcal{A}$ queries H on $(m_b^* \parallel \nu_{i^*})$ before ν_{i^*} is opened by an honest Rev, the reduction aborts; otherwise H_0 and H_1 are identical. Since ν_{i^*} has 256 bits of entropy, this abort probability is at most $\text{Adv}_{\mathcal{A}}^{\text{RO-guess}}$ (negligible under standard query bounds).

Game H_2 (hide RISE openings). Replace every RISE ciphertext E_j that opens a KHVC proof depending on h_{i^*} —in particular E_{i^*} and, after FileUp/KeyUp, every opening that has absorbed a challenge-dependent delta—by $\text{RISE.Enc}_k(U_j)$ for independent uniform $U_j \in \mathbb{G}_1$. A standard hybrid over the finitely many such openings reduces to RISE IND-ENC in $\mathbb{G}_1$, which holds under SXDH [4]. Call the resulting adversary $\mathcal{B}_1$.

Game H_3 (hide the body). Replace the Ring-LWR body of slot i^* by independent uniform elements of R_p (same length). By the DEM hop of Remark 2, this is decisional Ring-LWR for the current epoch secret (and, after KeyUp, for the updated secret). Call the resulting adversary $\mathcal{B}_2$.

Game H_4 (hide the stub). $\mathcal{A}$ still sees the stub C and the RISE openings of every slot $j \neq i^*$ (which, after H_2 , encrypt values independent of h_{i^*}). By Theorem 2, the two worlds that differ only in the challenge slot message (and hence only in h_{i^*}) are statistically identical except with probability at most n/p over setup ($\tau \in \Omega$). Thus H_4 is independent of b .

The Type-3 pairing check

$$e(C - [h]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega]_2)$$

uses only public $\mathbb{G}_2$ SRS elements $[1]_2, [\tau]_2$; nothing is extracted from $\mathbb{G}_2$ beyond what SXDH already assumes. Summing the hops (and the outer hybrid over $|\mathcal{I}| \leq n$ slots) yields the claimed bound. In H_4 the adversary's view is independent of b , so $\text{Adv}_{\mathcal{A}}^{\text{IND-DD-UP}}$ is at most the sum of the hop advantages. $\square$

Lemma 4 (IND-FileUp-CPA). *Any PPT adversary $\mathcal{A}$ against IND-FileUp-CPA satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-FileUp-CPA}} \leq \text{Adv}_{\mathcal{A}}^{\text{RO-guess}} + \text{Adv}_{\mathcal{B}_1}^{\text{RISE-ENC}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p}.$$

Proof. We follow [12, Thm. 4]. The adversary first obtains an honestly stored repository, then issues a challenge FileUp at slot i^* for two equal-length payloads (m'_0, m'_1) . The FileUp delta is one-hot: only coordinate i^* is nonzero.

Game H_0 (real). The challenger runs an honest FileUp for m'_b , producing a fresh nonce ν' , commitment $h' = \text{COM}(m'_b; \nu')$, one-hot openings $\{\Lambda_{\delta,j}\}_j$ by Lemma 2, RISE encryptions of those openings, a fresh tweak, and a fresh Ring-LWR body under the current DEM secret s .

Game H_1 (hide COM). Replace h' by a uniform $\mathbb{F}_p$ element, programming $H(m'_b \parallel \nu') = h'$ unless $\mathcal{A}$ queried that point. The hop is identical to H_0 unless the RO-guess event of Lemma 3 occurs.

Game H_2 (hide openings and body). Replace every $\text{RISE.Enc}_k(\Lambda_{\delta,j})$ by an encryption of a uniform $\mathbb{G}_1$ element (RISE IND-ENC / SXDH; hybrid over n openings), and replace the fresh body by uniform R_p elements. The new tweak ensures that every $a_b = H(\text{tweak}' \parallel b)$ is a fresh oracle string, so the body hop is Ring-LWR under the current s (Remark 2). The one-hot closed form of Lemma 2 produces the same opening group elements as a dense all-open; the reduction does not depend on which algorithm computes them.

Game H_3 (hide the updated stub). The updated stub is $C' = C \oplus C_\delta$ with C_δ committing to a one-hot vector whose only nonzero entry is the (already uniform) $h' - h_{i^*}$. Apply Theorem 2 to C' and to the openings of all slots except i^* : the view is statistically independent of b up to an n/p setup failure.

The same Type-3 pairing check as in Lemma 3 replaces any symmetric-group extraction step of [12]. $\square$

Lemma 5 (IND-REENC-CPA). *Any PPT adversary $\mathcal{A}$ against IND-REENC-CPA that triggers at most $t \leq 127$ body updates satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-REENC-CPA}} \leq \text{Adv}_{\mathcal{B}_1}^{\text{RISE-UPD}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p},$$

where $\mathcal{B}_1$ attacks RISE rand-IND-UPD in $\mathbb{G}_1$ and $\mathcal{B}_2$ attacks decisional Ring-LWR for the post-update DEM secret.

Proof. We follow [12, Thm. 5], separating header/stub unlinkability from body unlinkability.

Headers and stub. An honest KeyUp publishes $(\Delta_R = k'/k, Y' = k'g_1, \rho_0)$ and the server (i) applies RISE update with token Δ_R to every opening ciphertext, then (ii) adds $\text{RISE.Enc}_{k'}(\rho_0 W_j)$ to opening j and $[\rho_0 Z(\tau)]_1$ to the stub. Step (i) is exactly the RISE rand-IND-UPD game in $\mathbb{G}_1$ (SXDH) [4]: after the update, each opening ciphertext is distributed as a fresh encryption under k' of the same $\mathbb{G}_1$ message. Step (ii) is a public KHVC rerandomization: adding $\rho_0 Z$ does not change any evaluation on Ω , and by Theorem 2 (applied to a zero-message vector with blinding ρ_0) the updated stub together with the updated openings is statistically independent of the pre-KeyUp blinding, up to an n/p setup failure. A hybrid that first replaces RISE updates by fresh encryptions under k' , then replaces the stub/opening rerandomization by an independent fresh KHVC commitment to the same hash vector, is therefore indistinguishable from the real KeyUp header except for $\text{Adv}_{\mathcal{B}_1}^{\text{RISE-UPD}} + n/p$.

Bodies. The DEM token is $\Delta = s' - s \in R_q$. The server replaces every body block c by $c + F_\Delta(b)$. As in Remark 2, for $t \leq 127$ the resulting body equals a fresh encoding under s' plus a message-independent rounding error of infinity norm at most t . Replacing every updated body by uniform R_p elements is decisional Ring-LWR under s' . The public Δ does not help: it is uniform given s' , and F_Δ is already computable by the server. This hop is $\text{Adv}_{\mathcal{B}_2}^{\text{RLWR}}$.

Challenge bit. After both hops, the post-KeyUp repository (headers, stub, and bodies) is distributed as a fresh Store under (k', s') of the same plaintexts (up to the statistical n/p loss on the stub), independent of any pre-KeyUp challenge bit that IND-REENC asks to hide. Summing yields the bound. $\square$

Lemma 6 (OF-PTXT under key leakage). *OF-PTXT holds by KHVC binding and COM binding in the ROM, even if the current RISE key, the DEM secret, and every KeyUp nonce ρ_0 are leaked. Updating the body is not what the integrity argument uses.*

Proof. The adversary of [12, Thm. 6] may hold the RISE key, the DEM secret, and every KeyUp nonce ρ_0 , and may drive FileUp and KeyUp. A win is a repository that verifies at slot i for a plaintext that is not the latest honest file at i .

After an honest FileUp the stub is $C' = C + C_\delta$ and the honest opening at i verifies for h'_i . A stale plaintext whose commitment is $h_i \neq h'_i$, together with an opening Λ^* that verifies for C' , is two accepting openings of one slot of C' to two messages. That is a KHVC binding break, and the reduction in the binding theorem turns it into q -SDH. If the stale bytes hash to h'_i , the same transcript is a random-oracle collision for COM. The reduction does not need the RISE key.

KeyUp adds $[\rho_0 Z(\tau)]_1$ to the stub and $\rho_0 W_j$ to each opening. Both are multiples of Z , so every evaluation on Ω is unchanged: an opening that verified the pre-KeyUp stub for h_i still verifies the post-KeyUp stub for the same h_i . Thus KeyUp alone cannot create an OF-PTXT forgery; the binding argument above applies to FileUp (or to any adversarial repository that claims a plaintext other than the latest FileUp-accepted version at slot i). Knowledge of ρ_δ lets the adversary recompute C_δ . It does not let the adversary replace the stub the client stores. $\square$

6 Implementation and Evaluation

In this section, we present the implementation details of `uss-khvc` and conduct a comprehensive evaluation of its constituents, comparing our KHVC backend, Ring-LWR DEM, and full USS system against existing baselines.

6.1 Implementation Details

We implement `uss-khvc` as a unified Rust crate built upon `arkworks` 0.5 (for BLS12-381 pairings and polynomial operations), `aes-gcm` (for baseline symmetric operations), and `curve25519-dalek` 4.1 (for Ristretto group arithmetic).

Core Architecture. File bodies in `uss.rs` are processed using the Ring-LWR DEM with cached a_b NTTs. During key updates (**KeyUp**), the server updates every block by adding F_Δ directly, avoiding the overhead of re-computing a_b . Single-slot updates (**FileUp**) leverage our closed-form $O(1)$ `open_one_hot` routine, while dense updates employ FK20 for multi-opening proofs. To optimize performance, the crate dynamically selects the naive all-open algorithm for vector dimensions $n < 1024$, and switches to FK20 for $n \geq 1024$. The KHVC backend utilizes `Radix2EvaluationDomain` IFFTs alongside a custom $\mathbb{G}_1$ Cooley–Tukey FFT implementation.

Baselines and Comparison Targets. For comparison, we implement Catalano–Fiore HVC (CF-HVC) over Type-3 pairings, storing elements $h_i \in \mathbb{G}_1$, $\hat{h}_i \in \mathbb{G}_2$, and cross-terms $h_{i,j} \in \mathbb{G}_1$. We also evaluate existing Updatable Encryption (UE) schemes: SHINE [19], ReCrypt [20], and nested AES, using published reference numbers for the RLWE-based KH-PRF of [7] only as a qualitative reference (we do not re-benchmark that codebase here). Because RISE lacked a public implementation, we developed a Rust implementation within the same codebase to ensure consistent and fair benchmarking.

6.2 Experimental Setup

All benchmarks were executed on an Apple M5 Pro machine equipped with 24 GB RAM and 15 threads, running macOS 26.6.2 and `rustc` 1.97.1 with link-time optimization (`lto`) enabled.

Microbenchmarks for UE schemes were conducted via `fairbench` (`experiments/results/fair_ue.csv`) and `rlwrbench` (`experiments/results/rlwr_ue.csv`). Reported latencies reflect median values over multiple runs (21 iterations for headers and 4 KB files; 7 for 64 KB; 3–5 for 1 MB files) after discarding initial warm-up runs. Large file evaluations (100 MB) for ReCrypt are extrapolated at $100\times$ its 1 MB median, whereas slow algebraic schemes (e.g., RISE, SHINE0) are omitted at 100 MB due to high latency (1 MB RISE encryption takes 2.6 s).

6.3 HVC Analysis: Linear vs. Quadratic CRS

Table 1 compares the storage and computational overhead of KHVC against Catalano–Fiore HVC (CF-HVC) across varying vector sizes n .

Table 1: Microbenchmarks comparing KHVC and Catalano–Fiore HVC (CF-HVC). Timings are reported in milliseconds. The CRS column measures verification SRS (`crs.bytes`), excluding $\{W_j\}$ and including one 48-byte $[Z(\tau)]_1$. CF-HVC is omitted for $n \geq 4096$ due to prohibitive memory requirements.

n	Scheme	Verification CRS	Setup	Com	Open+Ver	All-Open (FK20 / Naive)
64	KHVC	3.3 KB	17	0.86	2.20	62 ms / 36 ms
64	CF-HVC	204 KB	298	3.86	5.22	259 ms
256	KHVC	12.3 KB	79	1.03	2.24	326 ms / 216 ms
256	CF-HVC	3.05 MB	4475	15.5	16.8	3995 ms
1024	KHVC	48.3 KB	379	2.36	3.45	1624 ms / 1694 ms
1024	CF-HVC	50.5 MB	70249	61.8	63.0	63131 ms
4096	KHVC	192 KB	1734	6.39	7.07	7707 ms
16384	KHVC	768 KB	8200	17.9	18.0	36495 ms

SRS Size Reduction. At $n = 1024$, CF-HVC requires a 50.5 MB verification CRS, whereas KHVC requires only 48.3 KB—a $1070\times$ reduction. When accounting for full operational parameters (including KeyUp data), KHVC consumes 96.3 KB, yielding a $512\times$ parameter reduction. At $n = 16384$ (2^{14}), CF-HVC’s quadratic scaling would require 12.8 GB of CRS, making it impractical for realistic deployments, whereas KHVC requires only 768 KB (and ≈ 1.5 MB operational parameters).

Update Performance. Performing KeyUp using precomputed Z -openings takes 63 ms at $n = 1024$ and 1.10 s at $n = 16384$, compared to 1.62 s and 36.5 s for a full FK20 all-open execution.

6.4 Payload Updatable Encryption Performance

Because algebraic schemes operate block-wise over group elements, their throughput characteristics differ significantly from symmetric ciphers. We decouple small header operations (1 group element) from bulk payload updates.

Table 2: Header UE latency (single group element) in median milliseconds. Tokens are compact and independent of file size.

Scheme	$ m $	Enc	Upd	Dec	Token Size	Expansion ($ ct / m $)
RISE ($\mathbb{G}_1$)	48 B	0.24	0.36	0.12	80 B	2.00
SHINE0 (Ristretto)	32 B	0.027	0.027	0.038	32 B	1.00

Header vs. Bulk Workloads. For sub-millisecond header updates (Table 2), RISE and SHINE0 provide efficient $O(1)$ token updates. However, for bulk payloads (Table 3), algebraic UE schemes suffer from substantial latency overhead: on a 1 MB file, AES-GCM achieves 241 MB/s (4.16 ns/B), whereas SHINE0 (1.93 MB/s), EC-ReCrypt (1.29 MB/s), and RISE (0.40 MB/s) are up to two to three orders of magnitude slower. Although nested AES achieves high throughput, decryption costs scale linearly with the number of key updates ($O(t)$), rendering it unsuitable over long operational lifetimes.

6.5 Ring-LWR DEM Evaluation

To accommodate bulk payload updates efficiently, we employ a Ring-LWR Data Encapsulation Mechanism (DEM) whose worst-case noise pad supports $t \leq 127$ consecutive updates ($N = 1024$,

Table 3: Bulk payload UE throughput (MB/s). Algebraic schemes operate block-wise. KHVC-USS combines the Ring-LWR DEM (Table 4) for bodies with RISE for openings.

Scheme	Payload ($ m $)	Enc	Upd	Dec	Enc Cost (ns/B)	Expansion ($ ct / m $)
RISE ($\mathbb{G}_1$)	4 KB	0.41	0.27	0.76	2428	2.02
SHINE0	4 KB	1.99	1.97	1.41	503	1.00
ReCrypt-KHPRF (EC)	4 KB	1.28	1.28	7.25	782	2.00
AES-256-GCM	4 KB	237	—*	239	4.21	1.01
Nested AES-GCM	4 KB	238	231	118 [†]	4.20	1.01
RISE ($\mathbb{G}_1$)	1 MB	0.40	0.26	0.73	2490	2.00
SHINE0	1 MB	1.93	1.92	1.37	519	1.00
ReCrypt-KHPRF (EC)	1 MB	1.29	1.30	7.52	776	2.00
AES-256-GCM	1 MB	241	—*	241	4.16	1.00
Nested AES-GCM	1 MB	247	246	125 [†]	4.05	1.00

* AES-GCM lacks a compact server token; updates require client re-encryption (240 MB/s).

[†] Decryption measured after a single key rotation epoch (two nested AES layers).

$q = 998244353$, $p = 2^{16}$). Eight bits per coefficient are reserved for noise, absorbing a maximum cumulative absolute error of 127.

Table 4: Single-threaded performance of the Ring-LWR DEM (excluding public matrix a_i generation).

Payload Size ($ m $)	Enc (MB/s)	Upd (MB/s)	Dec (MB/s)	Enc Cost (ns/B)	Expansion ($ ct / m $)
4 KB	33.4	33.8	34.4	30.0	2.00
64 KB	33.0	32.8	33.7	30.3	2.00
1 MB	32.6	32.6	32.9	30.7	2.00

Noise Growth and Concrete Security. Empirical verification across 64 blocks showed maximum absolute rounding errors of 42 after 64 updates and 78 after 128 updates. The 128-update trial is an *observed* average-case stress test: the worst-case analytic bound $\|e\|_\infty \leq t$ reaches 128 on the 128-th update and would overflow the 8-bit pad, so the guaranteed lifetime remains $t \leq 127$ (where $|e| \leq 127$ still fits). Both measured peaks stay strictly below 127. Evaluated using the `lattice-estimator` [10] (commit 53da598) with $m = 2^{20}$ samples, the most efficient finite attack (BDD) requires $\approx 2^{319}$ operations (uSVP requires $\approx 2^{322}$, dual attack $\approx 2^{334}$), confirming that these parameters offer security well above the 2^{128} threshold.

Caching Optimizations. Caching NTT representations of public polynomials a_b requires $4\times$ the plaintext size in memory, but reduces key rotation time for 4×1 GB files from 404 s (rebuilding a_b on the fly) to 254 s. The KeyUp token is 4240 bytes: $\Delta \in R_q$ (4096 B), RISE (Y', Δ_R) (48 + 32 B), the KHVC blinding ρ_0 (32 B), and a 32-byte epoch/counter word.

6.6 End-to-End USS Performance

We evaluate the complete KHVC-USS protocol against previous baselines across parameter generation, key updates, and file updates.

Partial File Updates (FileUp). As shown in Table 6, for single-slot updates ($k = 1$), our one-hot evaluation proof (Lemma 2) cuts opening latency from 1.62 s (using FK20) down to 194 ms. Relative to CF-HVC, however, $k = 1$ and $k = 8$ favor the baseline’s estimated Open+Ver cost (≈ 62 ms and ≈ 0.49 s): a KHVC one-hot FileUp must refresh all $n - 1$ off-slot openings

Table 5: Comparison between ASIACRYPT 2023 CF-USS and KHVC-USS.

Metric	2023 CF-USS	KHVC-USS (Ours)
Verification CRS ($n = 1024$)	50.5 MB	48.3 KB (1070 $\times$ reduction)
Operational CRS ($n = 1024$)	50.5 MB	96.3 KB (512 $\times$ reduction)
Parameter Setup (ParGen, $n = 1024$)	70.2 s	0.38 s (185 $\times$ speedup)
CRS Size ($n = 16384$)	12.8 GB	768 KB (Verification) / 1.5 MB (Operational)
Key Update (KeyUp, 4×1 GB)	Out of Memory	254 s / 4240 B token

Table 6: Server opening verification latency for FileUp modifying k slots at $n = 1024$.

k Slots	CF-HVC (Estimated)	KHVC (Measured)	Algorithmic Source
1	≈ 62 ms	194 ms	One-hot closed-form (KHVC)
8	≈ 0.49 s	1.67 s	FK20 multi-open (KHVC)
64	≈ 3.9 s	1.69 s	FK20 multi-open (KHVC)
1024	63.1 s	1.67 s	FK20 multi-open (38 $\times$ speedup)

($\Theta(n)$ scalar multiplications in $\mathbb{G}_1$), whereas the CF column reports only a single-slot Open+Ver estimate and does not charge a full server-side opening refresh. KHVC pulls ahead once k is large enough that CF’s per-slot work accumulates: the measured crossover is $k^* \approx 27$, and at $k = n = 1024$ KHVC is 38 $\times$ faster (1.67 s vs. 63.1 s). For dense updates ($k \geq 8$) FK20 keeps KHVC near-constant at ≈ 1.67 s.

Table 7: Key update benchmark comparison across identical repository storage configurations.

Scheme	Repository Size	KeyUp Latency	Data Transferred	OF-PTXT Integrity
KHVC-USS	4×1 GB	254 s	4240 B	Yes
AES Client Re-enc	4×1 GB	17.9 s [‡]	4.0 GB	No
Nested AES Upd	4×1 GB	17.5 s [‡]	4.0 GB	No
ReCrypt-KHPRF (EC)	4×1 GB	55 min [‡]	8.0 GB	No
SHINE0 on bodies	4×1 GB	37 min [‡]	4.0 GB	No
RISE on bodies	4×1 GB	4.6 h [‡]	8.0 GB	No
KHVC-USS	8×100 MB	58.9 s	4240 B	Yes
AES Client Re-enc	8×100 MB	3.5 s [‡]	800 MB	No
KHVC-USS	64×1 MB	2.22 s	4240 B	Yes
AES Client Re-enc	64×1 MB	0.28 s	64 MB	No

[‡] Extrapolated baseline figures: $T = T_{1\text{MB}} \cdot n \cdot |m|/1\text{MB}$.

Repository Key Rotation Trade-Offs. Table 7 highlights the primary trade-offs of KHVC-USS during key rotation of a 4×1 GB repository. While raw client-side AES re-encryption is faster (17.9 s), it requires transmitting the full 4.0 GB payload over the network and lacks server-side out-of-date plaintext integrity (OF-PTXT). In contrast, KHVC-USS completes key rotation on the server in 254 s while requiring only a 4240-byte transfer over the network—about an 8.7 $\times$ wall-clock improvement over the extrapolated SHINE0-on-bodies baseline (37 min) and far larger against RISE-on-bodies (4.6 h), noting that those algebraic figures are scaled from 1 MB microbenchmarks rather than measured end-to-end on 4 GB. As summarized in Table 8, processing time scales linearly with total payload size under the Ring-LWR DEM.

Table 8: Full KHVC-USS execution benchmarks across repository dimensions and payload sizes (with cached a_b).

n Files	File Size ($ m_i $)	ParGen	Store	Rev	FileUp	KeyUp
8	4 KB	3.9 ms	8.0 ms	1.9 ms	4.8 ms	7.5 ms
32	4 KB	7.6 ms	34.5 ms	2.0 ms	19.5 ms	26.3 ms
64	1 MB	16.5 ms	6.08 s	38 ms	0.15 s	2.22 s
8	100 MB	1.6 ms	80.9 s	7.82 s	14.4 s	58.9 s
4	1 GB	1.0 ms	425 s	79.7 s	192 s	254 s

7 Conclusion

In this paper, we presented KHVC-USS, a scalable Updatable Secure Storage system that eliminates the quadratic CRS storage bottleneck inherent in previous HVC-based constructions. By instantiating a key-homomorphic vector commitment over Type-3 BLS12-381 pairings, KHVC reduces operational parameters by $512\times$ at $n = 1024$ and compresses the verification SRS to $O(n)$, making large-scale secure storage practical. Theoretically, our construction guarantees statistical position hiding (up to an n/p setup bound) and preserves out-of-date plaintext authenticity (OF-PTXT) even under secret key compromise, without requiring an authenticated DEM. For bulk file updates, our Ring-LWR DEM provides provable IND-CPA confidentiality across 127 update epochs, with concrete generic-lattice estimator analysis placing the lowest-cost attack at $\approx 2^{319}$ operations—well above the 2^{128} security threshold.

Empirically, our single-slot FileUp algorithm achieves a 194 ms latency at $n = 1024$ using closed-form one-hot opening proofs. For a repository of four 1 GB files, end-to-end key updates complete in 254 s on the server with a compact 4240-byte network transfer. Overall, KHVC-USS demonstrates that integrity-aware, key-updatable secure storage can scale efficiently to real-world datasets without compromising on cryptographic security guarantees.

References

- [1] D. Boneh, X. Boyen. Short signatures without random oracles. EUROCRYPT 2004.
- [2] D. Catalano, D. Fiore. Vector commitments and their applications. PKC 2013.
- [3] A. Kate, G.M. Zaverucha, I. Goldberg. Constant-size commitments to polynomials and their applications. ASIACRYPT 2010.
- [4] A. Lehmann, B. Tackmann. Updatable encryption with post-compromise security. EUROCRYPT 2018.
- [5] A. Everspaugh, K. Paterson, T. Ristenpart, S. Scott. Key rotation for authenticated encryption. CRYPTO 2017. Implementation: <https://github.com/samscott89/ReCrypt>.
- [6] C. Boyd, G.T. Davies, K. Gjøsteen, Y. Jiang. Fast and secure updatable encryption. CRYPTO 2020, ePrint 2019/1457. Implementation: <https://github.com/YaoJGalteland/SHINE>.
- [7] D. Boneh, S. Eskandarian, S. Kim, M. Fisch. Improving speed and security in updatable encryption schemes. ASIACRYPT 2020, ePrint 2020/222. Implementation: https://github.com/moshih/UpdateableEncryption_Code.
- [8] D. Boneh, K. Lewi, H. Montgomery, A. Raghunathan. Key homomorphic PRFs and their applications. CRYPTO 2013.
- [9] A. Banerjee, C. Peikert, A. Rosen. Pseudorandom functions and lattices. EUROCRYPT 2012.

- [10] M.R. Albrecht, R. Player, S. Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology* 9(3), 2015. Code: <https://github.com/malb/lattice-estimator>.
- [11] National Institute of Standards and Technology. Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, 2020. Table 1: symmetric data-encryption keys, originator-usage period < 2 years.
- [12] L. Chen, H. Guo, Y.-N. Li, Q. Tang. Efficient secure storage with version control and key rotation. ASIACRYPT 2023, ePrint 2023/1421.
- [13] M. Campanelli, D. Fiore, N. Greco, D. Kolonelos, L. Nizzardo. Incrementally aggregatable vector commitments. ASIACRYPT 2020.
- [14] D. Boneh, B. Bünz, B. Fisch. Batching techniques for accumulators. CRYPTO 2019.
- [15] D. Feist, D. Khovratovich. Fast amortized KZG proofs. ePrint 2023/033 (FK20/FK22).
- [16] N. Fleischhacker, M. Hall-Andersen, M. Simkin. Extractable witness encryption for KZG commitments. ASIACRYPT 2024.
- [17] S. Setty, J. Thaler, R. Wahby. Unlocking the lookup singularity with Lasso. EUROCRYPT 2024.
- [18] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. ICALP 2018.
- [19] <https://github.com/YaoJGalteland/SHINE>
- [20] <https://github.com/samscott89/ReCrypt>
- [21] https://github.com/moshih/UpdateableEncryption_Code